



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 1

Integrante(s): 120002850 323064039

323071378 323214636

426059576

Semestre: 2027-1

Fecha de entrega: 1-Septiembre-2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	1
2. Marco Teórico	1
2.1. Paradigma Orientado a Objetos e Inmutabilidad de Entidades	1
2.2. Arquitectura de Interfaces Gráficas y Programación Dirigida por Eventos	1
2.3. Modelo de Renderizado Vectorial y Rasterización en Lienzo	2
2.4. Patrón Memento y Estructuras de Datos para Reversión de Estado	2
2.5. Flujo de Control y Aritmética Computacional Defensiva	2
2.6. Ingeniería de Software Asistida por Modelos de Lenguaje	2
3. Desarrollo	3
3.1. Ejercicio 1: Arquitectura del Editor Gráfico Generado por LLM	3
3.2. Ejercicio 2: Formalización y Ejecución de la Lógica del Pizarrón	7
3.2.1. Estructura Central de Despacho	7
3.2.2. Operaciones Aritméticas	8
3.2.3. Casos excepcionales de indeterminación	8
4. Conclusión	10
Referencias	10

Índice de figuras

1.	Respuesta de LLM.	3
2.	Interfaz de usuario del editor gráfico con figuras geométricas trazadas.	4
3.	Evidencia visual del funcionamiento del mecanismo de reversión de estados.	5
4.	Imagen matricial exportada exitosamente desde el lienzo del editor.	6
5.	Fotografía del pizarrón con los algoritmos y diagramas colaborativos de clase.	7
6.	Diagrama de flujo consolidado del menú interactivo de la calculadora.	7
7.	Diagrama de flujo de la adición entera.	8
8.	Diagrama de flujo de la sustracción entera.	8
9.	Diagrama de flujo del producto aritmético.	9
10.	Diagrama de flujo de la potencia iterativa.	9
11.	Diagrama de flujo de división controlada.	9
12.	Diagrama de flujo de módulo manual.	9
13.	Diagrama de flujo de radicación con control de dominio.	10

1. Introducción

El desarrollo de software profesional exige comprender dos modelos de ejecución complementarios: el flujo secuencial determinista que gobierna la lógica algorítmica y el modelo reactivo sobre el que operan las interfaces gráficas de usuario. Pasar de una especificación abstracta a un programa que funcione implica manejar tanto los eventos que produce el usuario (por ejemplo, los movimientos del cursor sobre un lienzo) como las validaciones necesarias cuando una operación aritmética tiene restricciones de dominio.

Esta práctica se justifica por la necesidad de establecer una línea base de análisis técnico en dos frentes de trabajo. En primer lugar, se examina una solución generada mediante un modelo de lenguaje de gran tamaño (LLM) para un editor gráfico interactivo, analizando su organización orientada a objetos y su manejo del historial de cambios. En segundo lugar, se formaliza la lógica aritmética construida de forma colaborativa en el aula, implementando una calculadora de consola que atiende los casos excepcionales, como la división entre cero o la raíz de un número negativo. Ambas actividades preparan el terreno para comparar más adelante el código asistido por inteligencia artificial frente al desarrollo humano.

Para evaluar el desarrollo de estas competencias, se establecieron los siguientes objetivos:

- Analizar la arquitectura orientada a objetos de una herramienta gráfica interactiva generada por un LLM, examinando la respuesta obtenida en cuanto a su división en componentes, su manejo de eventos y su control del estado del dibujo.
- Implementar y verificar la lógica aritmética formulada en el pizarrón mediante una aplicación estructurada en Java, atendiendo los casos excepcionales necesarios para su correcto funcionamiento.

2. Marco Teórico

2.1. Paradigma Orientado a Objetos e Inmutabilidad de Entidades

La Programación Orientada a Objetos (POO) organiza el software mediante abstracciones que encapsulan estado y comportamiento [1]. Un principio relevante en el diseño actual es la inmutabilidad: un objeto inmutable conserva sus atributos tal como quedaron al momento de su creación, lo que elimina efectos secundarios y facilita el razonamiento sobre el programa. En Java, esta propiedad se consigue declarando la clase como `final` y sus campos como `final`, de modo que no admitan reasignación después de la construcción [1].

2.2. Arquitectura de Interfaces Gráficas y Programación Dirigida por Eventos

A diferencia de un programa secuencial, una interfaz gráfica opera bajo un modelo reactivo gobernado por un hilo de despacho de eventos. Según lo expuesto en [1], los periféricos generan interrupciones que el sistema traduce en objetos de evento, los cuales se entregan a los componentes registrados como escuchas. Ese registro corresponde al patrón de diseño *Observer*, en el que un sujeto notifica automáticamente a sus dependientes cada vez que cambia su estado [2]. Gracias a este mecanismo, el lienzo puede atender las transiciones del ratón y recalcular coordenadas sin bloquear el hilo de la interfaz [1].

2.3. Modelo de Renderizado Vectorial y Rasterización en Lienzo

El dibujo en pantalla comprende el cálculo de primitivas geométricas y su posterior rasterización. El contexto gráfico de la plataforma ofrece un búfer de dibujo sobre el cual se trazan líneas, rectángulos y óvalos, y permite aplicar transformaciones geométricas antes del trazado [3]. Para conservar el resultado fuera de la aplicación, la biblioteca estándar permite construir una imagen en memoria y escribirla en un formato matricial como PNG mediante las clases de entrada y salida de imágenes [3].

2.4. Patrón Memento y Estructuras de Datos para Reversión de Estado

La función de deshacer (*undo*) exige conservar el historial de los cambios aplicados. El patrón *Memento* atiende ese requerimiento capturando instantáneas del estado interno de un objeto sin exponer su representación, de forma que pueda restaurarse más tarde [2]. Desde el punto de vista de las estructuras de datos, ese historial se administra con una pila, es decir, una colección de acceso LIFO (*Last-In, First-Out*) cuyas operaciones de apilado y desapilado se resuelven en tiempo constante $O(1)$ [4].

2.5. Flujo de Control y Aritmética Computacional Defensiva

El diseño de algoritmos aritméticos se apoya en estructuras de control selectivas e iterativas, que permiten bifurcar el flujo según una condición o repetir un bloque un número determinado de veces [1]. La programación defensiva agrega un criterio adicional: toda precondition matemática debe validarse antes de ejecutar el cálculo.

- **División y Módulo:** el algoritmo de la división establece que, para todo entero a y todo entero $b > 0$, existen enteros únicos q y r , con $0 \leq r < b$, tales que $a = b \cdot q + r$ [5]. De aquí se desprende que el divisor nulo queda fuera del dominio de la operación.
- **Radicación:** la raíz cuadrada solo está definida en los números reales para operandos no negativos; en Java, la función correspondiente devuelve el valor especial **NaN** cuando el argumento es negativo, por lo que conviene validar la entrada antes de invocarla [3].
- **Potenciación:** la definición recursiva de la potencia con exponente entero no negativo fija el caso base $a^0 = 1$, condición que un ciclo acumulador debe contemplar de forma explícita [5].

2.6. Ingeniería de Software Asistida por Modelos de Lenguaje

Un modelo de lenguaje de gran tamaño (LLM) es un sistema entrenado sobre grandes volúmenes de texto y código que genera nuevas secuencias a partir de los patrones aprendidos; en los últimos años, este tipo de modelos se ha incorporado a tareas de generación, resumen y reparación de código [6]. Aun así, la literatura señala que la corrección funcional no es el único criterio de aceptación: el código producido requiere revisión humana en aspectos de calidad y mantenibilidad, como la cohesión y el acoplamiento, antes de integrarse a un proyecto real [6].

3. Desarrollo

3.1. Ejercicio 1: Arquitectura del Editor Gráfico Generado por LLM

Para el primer ejercicio se solicitó al LLM la generación de una herramienta gráfica modular en Java, estructurada en tres componentes: la ventana principal, el lienzo interactivo y el registro de figuras geométricas. El requerimiento entregado al modelo y la respuesta obtenida se muestran en la Figura 1.

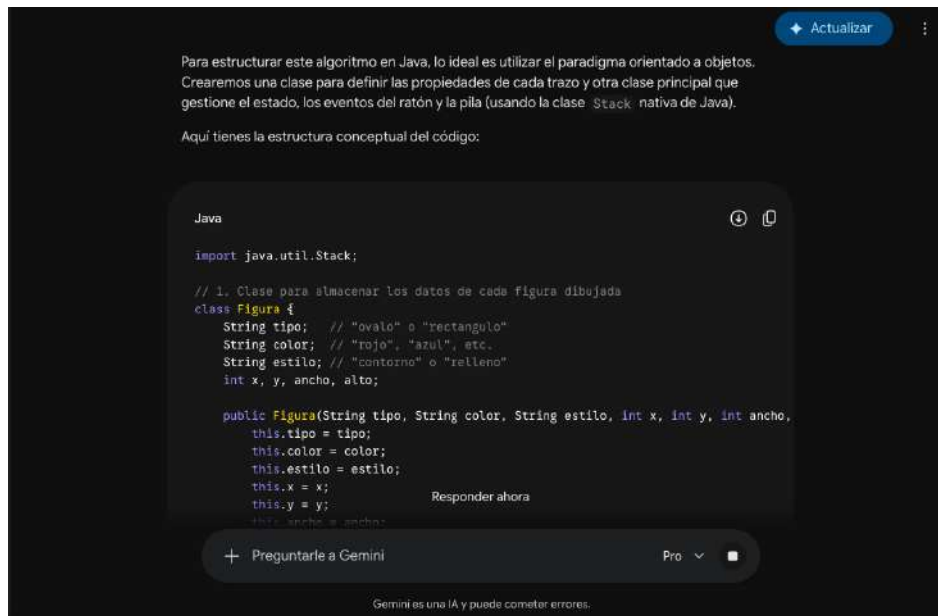


Figura 1: Respuesta de LLM.

1. **Modelado de Figuras:** la entidad geométrica se diseñó como un registro inmutable que almacena el tipo de figura, el punto de inicio, el punto de destino, el color de trazado y la bandera booleana de relleno. Esta inmutabilidad evita que una figura ya dibujada sea alterada por eventos posteriores del ratón.
2. **Captura de Eventos y Previsualización Dinámica:** en el panel del lienzo se integró un adaptador de ratón que registra la coordenada donde inicia la presión. Durante el arrastre, el programa genera una figura temporal de previsualización que se redibuja en tiempo real. Al liberar el botón izquierdo se comprueba que las dimensiones superen un umbral mínimo, con el fin de descartar trazos accidentales, y la figura definitiva se incorpora a la lista principal (Figura 2).
3. **Mecanismo de Deshacer (*Undo*):** antes de cada modificación de la lista de figuras, el lienzo genera una copia de la colección actual y la coloca en la cima de una pila construida sobre una cola de doble extremo. La operación de deshacer extrae la última instantánea disponible y repinta el lienzo, con lo que la reversión no destruye información, tal como se ilustra en la Figura 3.
4. **Exportación de la Imagen:** para conservar el dibujo se crea una imagen en memoria con fondo blanco, se trasladan las primitivas mediante el contexto gráfico bidimensional y el resultado se escribe en un archivo PNG en disco (Figura 4).

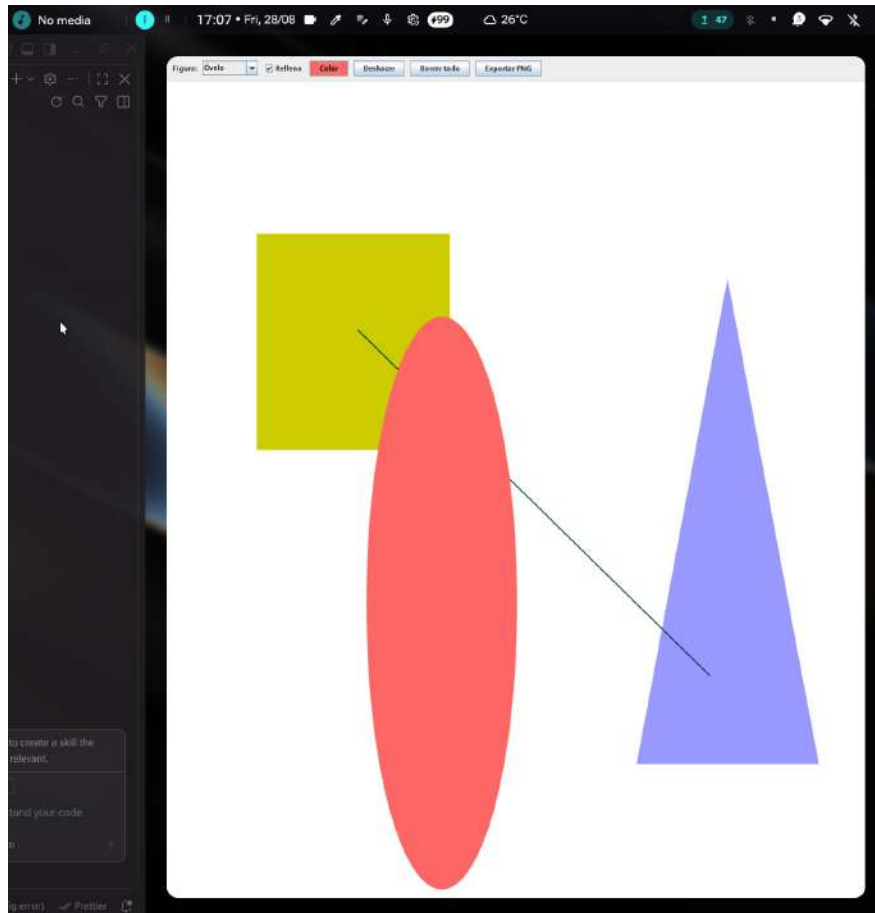


Figura 2: Interfaz de usuario del editor gráfico con figuras geométricas trazadas.

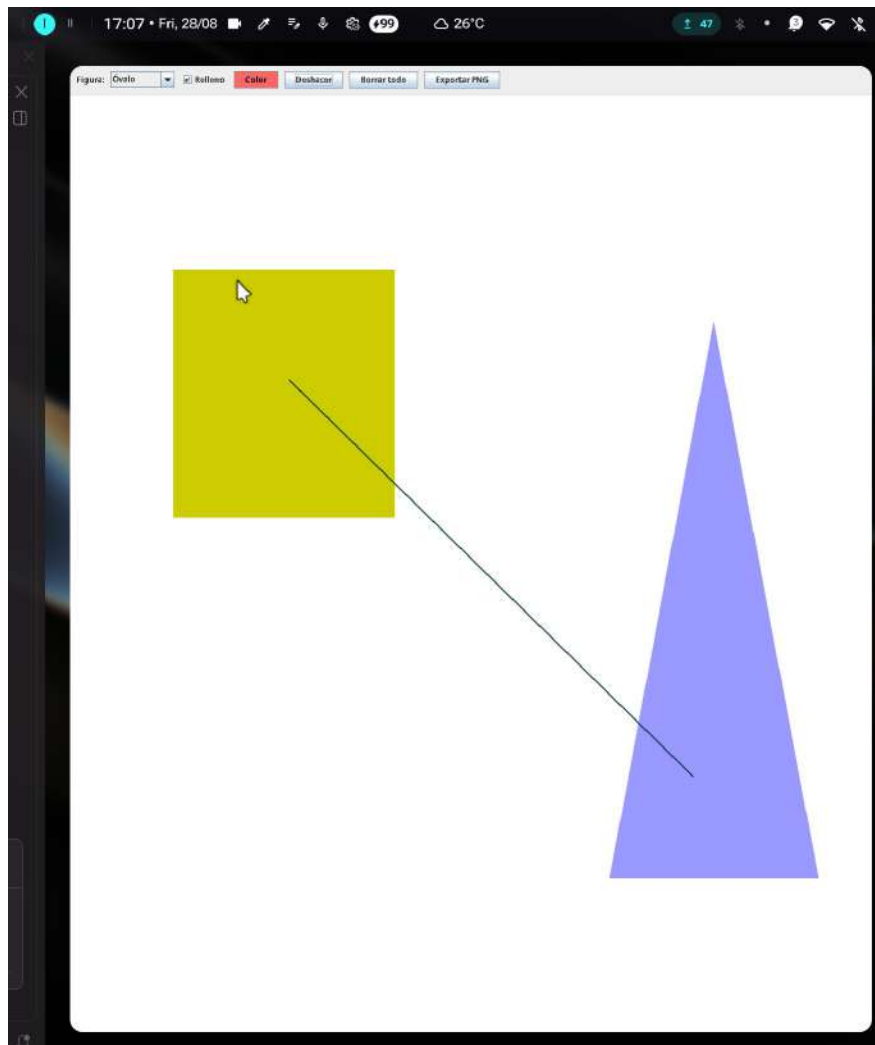


Figura 3: Evidencia visual del funcionamiento del mecanismo de reversión de estados.

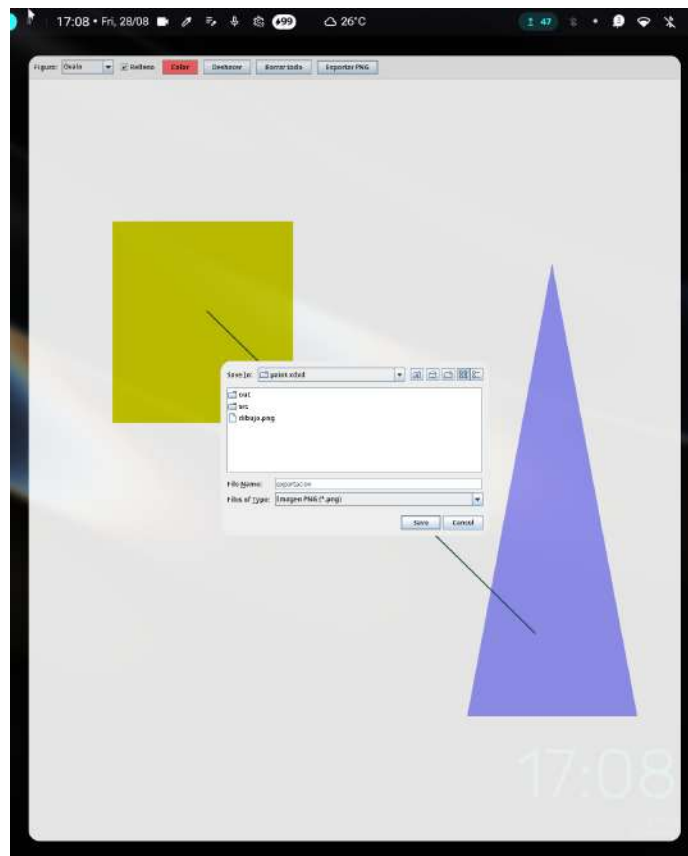


Figura 4: Imagen matricial exportada exitosamente desde el lienzo del editor.

3.2. Ejercicio 2: Formalización y Ejecución de la Lógica del Pizarrón

El segundo ejercicio consistió en traducir los algoritmos y diagramas de flujo elaborados en grupo en el pizarrón del aula (Figura 5) a un programa interactivo de consola en Java.

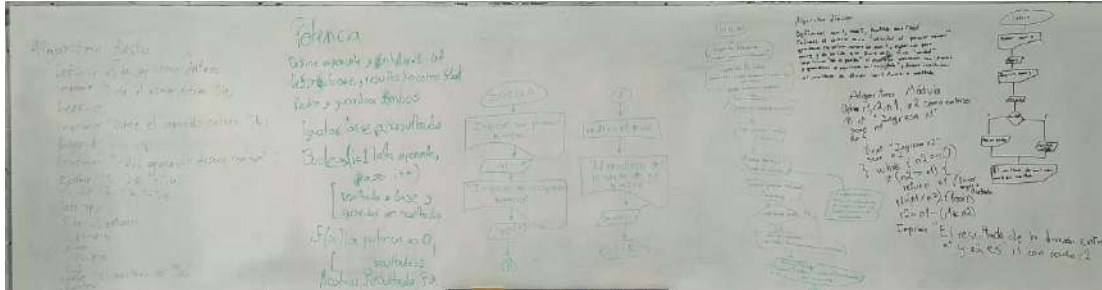


Figura 5: Fotografía del pizarrón con los algoritmos y diagramas colaborativos de clase.

3.2.1. Estructura Central de Despacho

El flujo principal del programa pide al usuario que seleccione la operación desde un menú numérico. Esa entrada se evalúa con una estructura de selección múltiple (*switch-case*), que dirige el control hacia la rutina correspondiente y regresa al menú al terminar (Figura 6).

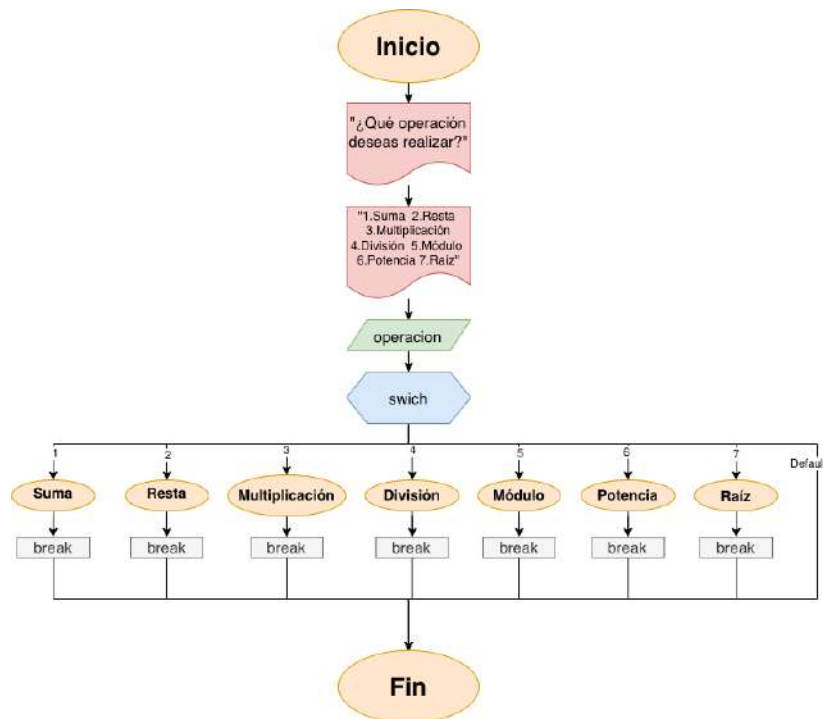


Figura 6: Diagrama de flujo consolidado del menú interactivo de la calculadora.

3.2.2. Operaciones Aritméticas

- **Adición y Sustracción:** procesan operandos enteros mediante evaluación directa (Figuras 7 y 8).
- **Multipliación:** opera con números de punto flotante de doble precisión, para admitir valores con decimales (Figura 9).
- **Exponenciación:** conforme al diseño del pizarrón, se prescindió de bibliotecas matemáticas externas y se implementó un ciclo que acumula el producto de la base tantas veces como indique el exponente entero, con una validación previa para el exponente cero (Figura 10).

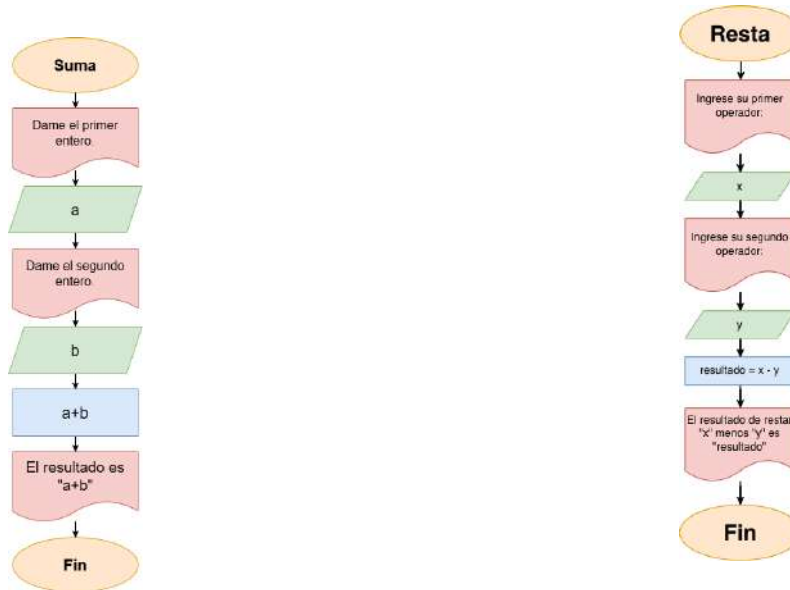


Figura 7: Diagrama de flujo de la adición entera.

Figura 8: Diagrama de flujo de la sustracción entera.

3.2.3. Casos excepcionales de indeterminación

Las operaciones restantes exigieron un tratamiento más cuidadoso de las condiciones de entrada:

- **División:** se incluye una validación antes del cálculo; si el denominador es cero, el programa muestra un aviso y suspende la operación, con lo que se evita la indeterminación (Figura 11).
- **Módulo Manual:** el residuo se obtiene calculando el cociente entero $q = \lfloor a/b \rfloor$ y restando el producto $b \cdot q$ al dividendo original, siempre que el divisor no sea nulo (Figura 12).
- **Radicación Real:** se verifica que el radicando sea no negativo antes de invocar la función de raíz cuadrada y, ante una entrada negativa, se informa que no existe solución en los números reales (Figura 13).



Figura 9: Diagrama de flujo del producto aritmético.

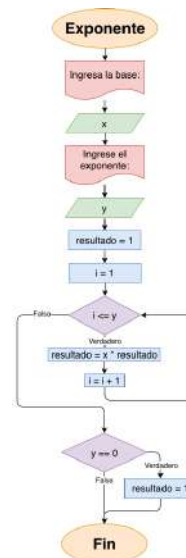


Figura 10: Diagrama de flujo de la potencia iterativa.

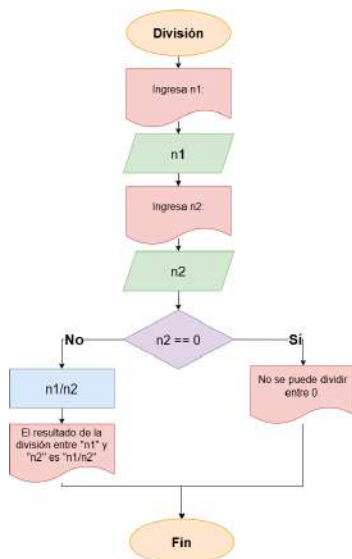


Figura 11: Diagrama de flujo de división controlada.

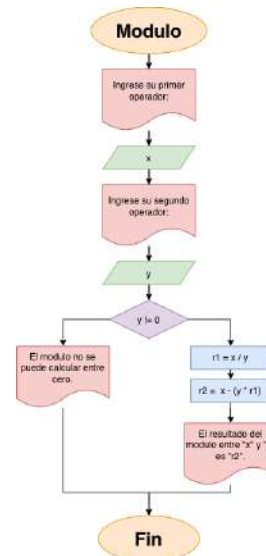


Figura 12: Diagrama de flujo de módulo manual.

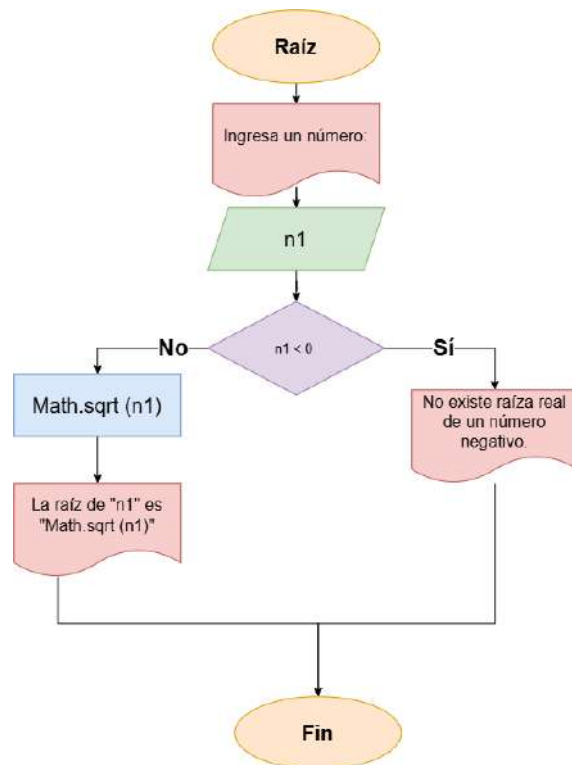


Figura 13: Diagrama de flujo de radicación con control de dominio.

4. Conclusión

La práctica permitió contrastar dos formas de trabajo dentro del desarrollo de software: la arquitectura reactiva de una interfaz gráfica y el modelado de algoritmos en consola. El análisis del editor gráfico mostró que un LLM puede producir un programa funcional y organizado en componentes; sin embargo, también evidenció que el código generado no siempre resulta fácil de seguir, ya que concentra varias responsabilidades en pocos métodos y omite comentarios en las partes menos evidentes.

En relación con los objetivos planteados en la introducción, se identifica el siguiente cumplimiento:

1. **Análisis de la arquitectura del editor gráfico generado por LLM:** *Cumplido.* Se revisó e ilustró la relación entre los componentes, la captura de coordenadas del ratón, la previsualización en tiempo real y la exportación de la imagen, lo que establece la línea base técnica para las comparaciones posteriores.
2. **Implementación y verificación de la lógica del pizarrón:** *Cumplido.* La calculadora integró todos los flujos diseñados por el grupo: bloquea la división entre cero y la raíz de números negativos mediante validaciones previas, y resuelve el residuo y la potencia con lógica propia, sin recurrir a operadores ni bibliotecas que hicieran el trabajo.

En conjunto, la práctica cumplió su propósito formativo al separar el análisis conceptual de la implementación. Los programas desarrollados dejan una base concreta para abordar, en la siguiente etapa del curso, la comparación entre el código generado por inteligencia artificial y el escrito por completo por desarrolladores.

Referencias

- [1] P. J. Deitel y H. M. Deitel, *Java How to Program: Early Objects*, 11.^a ed. Hoboken, NJ, EE. UU.: Pearson Education, 2017.
- [2] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, EE. UU.: Addison-Wesley, 1994.
- [3] H. Schildt, *Java: The Complete Reference*, 11.^a ed. Nueva York, NY, EE. UU.: McGraw-Hill Education, 2018.
- [4] R. Sedgewick y K. Wayne, *Algorithms*, 4.^a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley, 2011.
- [5] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8.^a ed. Nueva York, NY, EE. UU.: McGraw-Hill Education, 2019.
- [6] X. Hou *et al.*, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, n.^o 8, art. 220, pp. 1–79, dic. 2024.